

Business Process Remaining Time Prediction Based on Incremental Event Logs

Na Guo, Cong Liu, Qi Mo, Jian Cao, Chun Ouyang, Xixi Lu, and Qingtian Zeng

Abstract—Predictive Process Monitoring (PPM) aims to predict the future state of running process instances to enable timely interventions to mitigate potential risks. As one of the most fundamental tasks in PPM, process remaining time prediction focuses on preventing timeout occurrences. While various deep learning-based approaches have been developed for this purpose, they often rely on pre-established static prediction models and struggle to maintain accurate predictions when the process undergoes dynamic changes, such as an expanding sales channels. To tackle this challenge, this paper proposes an incremental process remaining time prediction framework by continuously updating the prediction model based on an incremental event log. Specifically, a feature selection strategy is first introduced to extract effective features from event logs. Leveraging effective features can significantly improve the prediction quality by capturing the changes in process information. Then, three incremental log-based updating mechanisms, including period-based updating, quantity-based updating, and concept-drift-based updating, along with a reconstruction strategy, are proposed to dynamically adjust the prediction model in response to business changes. Finally, LSTM, Transformer, and Auto-encoder models are adapted and integrated into the proposed framework. The approach has been implemented and publicly released. Experimental evaluation using nine real-life event logs demonstrate that the proposed framework and its three instantiations (i.e., LSTM-based, Transformer-based, and Auto-encoder-based ones) outperform state-of-the-art techniques in terms of prediction accuracy.

Index Terms—Predictive process monitoring, Remaining time prediction, Process mining, Incremental event log.

1 INTRODUCTION

PROCESS mining [1] refers to a set of techniques designed to extract valuable insights from historical event logs, enabling enterprises to enhance process efficiency and business competitiveness [2]. As a typical subfield of process mining, Predictive Process Monitoring (PPM) has garnered significant attention in recent years [3]. In general, PPM focuses on predicting the future states of ongoing process instances. Typical prediction tasks include remaining time prediction [4], process outcome prediction [5], next activ-

ity prediction [6], and next event duration prediction [7]. Among these, remaining time prediction for an ongoing process instance enables users to take proactive measures and adjust subsequent execution steps to mitigate risks associated with potential delays.

Traditionally, remaining time prediction relied on process models and statistical techniques that consider different states of control flow [8]. However, these approaches often struggled with the dynamic and complex nature of real-world processes, leading to limited prediction accuracy. Recently, deep learning approaches have been employed to enhance remaining time prediction [9]. Techniques such as Long Short-Term Memory (LSTM) networks, convolutional neural network (CNN), and Transformers can capture intricate patterns and dependencies within event logs, significantly improving prediction accuracy and adaptability to evolving process conditions. However, these approaches often rely on pre-established prediction models and fail to maintain accurate predictions when processes undergo dynamic changes.

In real-life business scenarios, processes evolve dynamically due to internal and external factors. Common factors include seasonal variations (e.g., transitions between off-season and peak season), enterprise development and organizational restructuring, personnel transfers, equipment upgrades, and emergencies. These factors are reflected in event logs as case-level or event-level attributes. Existing remaining time prediction techniques face several limitations when dealing with dynamic processes: 1) there is no systematic feature selection technique to extract effective features from event logs with an excessive number of attributes to build high-quality prediction models that capture input information accurately; and 2) effective updating mechanisms to support business process changes are missing for remaining

This work was partly supported by the National Key R&D Program of China under Grant 2022ZD0119501, National Natural Science Foundation of China under Grant 62472264 and 52374221, the Taishan Scholars Program of Shandong Province under Grants ts20190936 and Grant tsqn201909109, the Natural Science Excellent Youth Foundation of Shandong Province under Grant ZR2021YQ45, and the Youth Innovation Science and Technology Team Foundation of Shandong Higher School under Grant 2021KJ031. (Corresponding Authors: C. Liu and Q. Mo)

- N. Guo is with School of Electrical and Electronic Engineering, Shandong University of Technology, Zibo 255000, China. (e-mail: guona_7@163.com)
- C. Liu is with the NOVA Information Management School, Nova University of Lisbon, 1070-312 Lisbon, Portugal, and also with the School of Computer Science and Technology, Shandong University of Technology, Zibo 255000, China. (e-mail: cliu@novaims.unl.pt).
- Q. Mo is with the School of Software, Yunnan University, Kunming 650091, China. (e-mail: moqi@ynu.edu.cn)
- J. Cao is with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200030, China. (e-mail: caojian@cs.sjtu.edu.cn)
- C. Ouyang is with the School of Information Systems, Queensland University of Technology, Brisbane, Australia. (e-mail: c.ouyang@qut.edu.au)
- X. Lu is with the Department of Information and Computing Sciences, Utrecht University, Utrecht, the Netherlands. (email: x.lu@uu.nl)
- Q. Zeng is with the College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China. (e-mail: qtzeng@163.com)

time prediction in an incremental setting.

To handle the above limitations, we propose a feature-selection-based framework to predict business process remaining time using incremental event logs. The main contributions include:

- a wrapper-based feature selection strategy is applied to prepare effective feature combinations as input to construct high-quality prediction models; and
- three incremental log-based prediction model updating mechanisms, including period-based updating, quantity-based updating, and concept-drift-based updating, as well as a reconstruction strategy, are proposed to dynamically update the prediction model, enabling adaptation to evolving process.

The remainder of this paper is organized as follows. Section 2 presents a review of related work. Section 3 introduces the background knowledge. Section 4 presents the proposed process remaining time prediction framework based on incremental event logs. Section 5 evaluates the effectiveness and applicability of the proposed approach using nine real-life event logs. Section 6 discusses potential threats to the validity of the proposed approach. Section 7 concludes the paper and outlines future research directions.

2 RELATED WORK

PPM focuses on learning prediction models to provide accurate predictions for ongoing process instances, including future execution steps, remaining time, process outcome, and resource load [10]–[11]. In general, existing PPM techniques can be divided into two categories, i.e., process-aware techniques and non-process-aware ones [12]–[13].

Process-aware techniques predict the target value by first constructing process models manually or automatically from event logs, and subsequently combining with statistical techniques [8], [14]. For example, van der Aalst *et al.* proposed a remaining time prediction approach based on marked transition systems, and an independent prediction tool named FSM analyzer [15]. To enhance the expressiveness of discovered models, Rogge-Solti *et al.* used stochastic Petri nets with generally distributed transitions as the underlying prediction model to forecasting the remaining time of process instances [16]. On the one hand, these approaches may be easy to interpret and understand as an explicitly model exists. On the other hand, the prediction accuracy may be limited or even unstable, as the quality of manually constructed or automatically discovered process models can vary significantly depending on the underlying event logs.

Non-process-aware techniques typically use machine learning algorithms to construct prediction models [17]. With the improvement of data quality and quantity in event logs, deep learning models have been widely used recently [18]. Tax *et al.* applied LSTM for business process prediction in [19], incorporating date and week features extracted from timestamp attributes as input to the prediction model. Compared to process-aware approaches, deep learning techniques significantly improve prediction accuracy. Similarly, Camargo *et al.* used LSTM to build a prediction model integrating resource attributes into the input and adopting a word vector encoding approach [20]. These enhancements

led to superior prediction performance compared to conventional LSTM-based approaches. Instead of LSTM, Bukhsh *et al.* applied the Transformer model to improve the remaining time prediction accuracy in [21]. Duong *et al.* proposed to use graph neural network models for remaining time prediction, where the precedence relations among activities are represented as graphs [22]. Similarly, Amiri Elyasi *et al.* transformed event logs into graph datasets and leveraged graph-oriented data to train Process Graph Transformer Networks for predicting the remaining time of business process instances [23]. However, most of these techniques tend to learn highly complex models, which are often difficult to interpret. More recently, we introduced a cascade prediction framework to enhance remaining time prediction by establishing correlations between input features and their impact on prediction outcomes [24]. In addition, most existing studies focus on designing new algorithms to improve prediction accuracy, while overlooking the necessity of invoking prediction services. Cao *et al.* proposed a deep reinforcement learning-based strategy to dynamically decide the next prediction point for invoking remaining time prediction services in [25].

To the best of our knowledge, existing remaining time prediction approaches fail to adequately accommodate the dynamic nature of business processes. While incremental learning strategies have been explored in outcome prediction and next activity prediction [26], their application to remaining time prediction remains limited. Baier *et al.* systematically analyzed the effect of different data selection strategies on incremental learning based on outcome prediction [27]. Similarly, Rizzi *et al.* evaluated three strategies for periodic rediscovery and incremental prediction model construction [28]. These studies highlight the advantages of incremental updating in balancing prediction accuracy and computational efficiency compared to full retraining or static models. Focusing on process concept drift, Pauwels *et al.* updated the prediction model solely based on control-flow perspective [29]. Pasquadibiscegli *et al.* applied the ADaptive WINdow concept drift detection strategy to address the next activity prediction problem in dynamic business data streams [30]. Furthermore, Mangat *et al.* investigated the next activity prediction task for non-stationary processes with unseen data variability to accommodate changing process behaviors in [31]. However, in remaining time prediction, changes in other process attributes (beyond control-flow) can also degrade prediction model performance, highlighting the need for adaptive updating strategies tailored to dynamic process conditions.

3 BACKGROUND KNOWLEDGE

Basic concepts of event logs and process remaining time prediction are briefly reviewed in this section.

3.1 Event Logs

An event log is a set of cases where each case records one execution of the underlying business process. Each case is composed of a chronologically ordered events where each event refers to the execution of an activity.

Definition 1. (Event, Attribute) Let E be the event universe, i.e., the set of all event identifiers. Events may

TABLE 1
A Fragment of the Helpdesk Log

Case id	Event id	Complete Timestamp	Activity	Resource	Seriousness	...
1	e_1	2012/10/9 14:50	Assign seriousness	Peter	Level 1	...
	e_2	2012/10/9 14:51	Take in charge ticket	Peter	Level 1	...
	e_3	2012/10/12 15:02	Take in charge ticket	Mike	Level 1	...
	e_4	2012/10/25 11:54	Resolve ticket	Peter	Level 1	...
	e_5	2012/11/9 12:54	Closed	Niki	Level 1	...
...	...	...	...	...	...	...

be characterized by various attributes. Let AN be a set of attribute names. For any event $e \in E$ and attribute $n \in AN$: $\#_n(e)$ represents the value of attribute n for event e .

Let U_A be the activity universe, $\#_{act}(e) \in U_A$ is the activity name associated to event e .

Definition 2. (Case, Trace and Event log) Let C be the case universe. For any $c \in C$ and attribute name $n \in AN$: $\#_n(c)$ is the value of attribute n for case c . Each case has a mandatory attribute, i.e., trace, such that $\#_{trace}(c) \in E^*$. A trace is a finite sequence of events, i.e., $\sigma \in E^*$, such that each event appears only once and all events are ordered by the timestamp. An event log is a set of cases $L \subseteq C$ such that each event appears at most once in the entire log.

Definition 3. (Trace prefix) $tp^k(\sigma)$ is a trace prefix of σ , i.e., the sub-sequence consisting of the first k elements of σ .

Table 1 shows a fragment of the public Helpdesk event log¹, where one trace containing five events is given. Each event has a unique event id and several attributes. For example, event e_4 is an instance of activity “Resolve ticket” that occurred on October 25th, 2012 at 11:54. This event is executed by Peter, and its Seriousness is Level 1. $\sigma = \langle e_1, e_2, e_3, e_4, e_5 \rangle$ is the trace of case 1. $tp^3(\sigma) = \langle e_1, e_2, e_3 \rangle$ represents the prefix of trace σ with $k = 3$.

3.2 Process Remaining Time Prediction

Remaining time prediction aims to answer business questions such as “how long will my business process take to complete?”. Accurately predicting the remaining time ensures that ongoing process instances can be completed within predefined time constraints, enabling timely interventions without negatively impacting subsequent processes. The input to a remaining time prediction task consists of a set of trace prefixes, while the output is the estimated time required from the current moment until the completion of the process instance.

Consider a medical scenario as an example. Suppose a patient wants to determine how long their current medical treatment process will take to decide whether an important work meeting needs to be postponed. The patient’s symptoms and the number of patients in the hospital may affect the execution time and can be used as features to predict the remaining time.

1. <https://doi.org/10.4121/uuid:0c60edf1-6f83-4e75-9367-4c63b3e9d5bb>

4 INCREMENTAL PROCESS REMAINING TIME PREDICTION FRAMEWORK

This section first presents an overview of the approach and then provides a detailed explanation of each step in the incremental business process remaining time prediction framework.

4.1 An Approach Overview

Fig. 1 shows an overview of the proposed incremental business process remaining time prediction framework that consists of the following three steps:

- **Step 1: Event Log Pre-processing.** By taking as an input event log, an enriched event log is generated by extracting time-related attributes from timestamp data, as well as encoding activities and other attributes.
- **Step 2: Feature-selection-based Prediction Model Construction.** By taking an enriched event log as input, a feature selection strategy is proposed to extract an effective feature combination. The feature-selection-based remaining time prediction model is then constructed based on the extracted feature combination, and the initial prediction model is trained using historical event logs.
- **Step 3: Prediction Model Updating Strategy.** The remaining time of each event in a ongoing process instance is predicted in a real-time manner, and each trace is added to the incremental event log in chronological order. The prediction model is updated and reconstructed according to the incremental updating mechanism and the reconstruction strategy.

4.2 Event Log Pre-processing

Timestamp extension: To enrich time information, six numerical attributes that may have a potential impact on the remaining time prediction task, i.e., *duration*, *allDuration*, *month*, *day*, *week*, and *hour*, are derived from the timestamp attribute. Specifically, *duration* represents the time elapsed from the completion of the last event to the completion of the current event, and *allDuration* denotes the time elapsed from the beginning of the case to the completion of the current event.

Activity encoding: To fully capture the contextual relations among activities, the Continuous Bag of Words (CBoW) technique [32] is used to train activity representation vectors. CBoW is a neural network model that generates

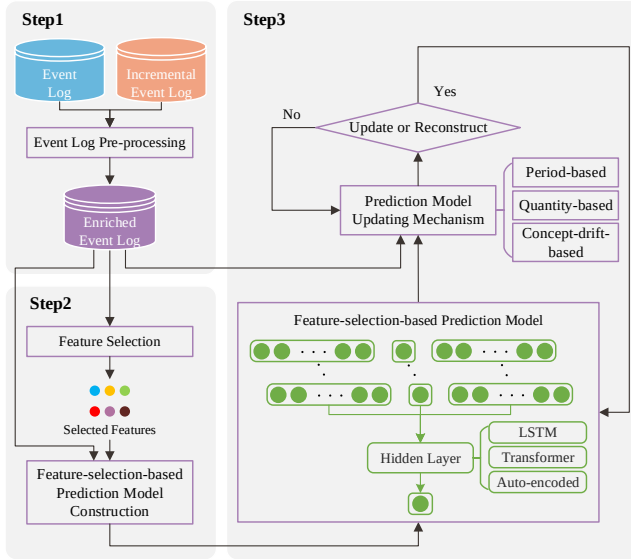


Fig. 1. An Approach Overview.

word vectors and predicts middle words based on surrounding words. In this study, CBoW is applied to activity encoding by predicting the next activity given a trace prefix.

Other attributes encoding: To represent categorical attributes, the embedding matrix is randomly initialized using embedding encoding techniques, as the contextual relationships among these attributes are not well-defined. The embedding matrix can either be continuously refined during the training of subsequent tasks to achieve a better representation or remain fixed as a simple unique category identifier [33]. For numerical attributes, min-max normalization is applied to unify the order of magnitude.

4.3 Feature-selection-based Prediction Model Construction

To enhance remaining time prediction, important positive features are first selected from the entire event log and used as input to the prediction model. This process eliminates irrelevant attributes, ensuring model simplicity and improving prediction efficiency. An effective feature selection strategy that incorporates both backward elimination and forward selection strategies is applied to filter out features that negatively impact prediction accuracy, resulting in an effective feature combination (EFC). This strategy automatically selects the optimal feature combination for prediction tasks based on different event logs. By setting an error threshold, features with minimal positive effects are removed to prevent over-fitting. In addition, the strategy constructs an incremental feature tree that offers valuable insights, such as feature importance ranking, each feature's contribution to the prediction task, and the relationships between features. This information enables users to construct a dedicated feature set that balances prediction accuracy and computational efficiency. For more details of the strategy, readers may refer to our previous work [24]. Then, by taking the EFC generated by the strategy as input, neural network units are selected based on the characteristics of business processes to construct a tailored feature-selection-based prediction model. Note that the input can be adjusted according

to the characteristics of the different neural network units, the hidden layer of the prediction model can be replaced by alternative neural network components, and the output is the remaining time of an ongoing process instance. Without loss of generality, LSTM [34], Auto-encoder [35], and Transformer [21] are adapted to instantiate the proposed prediction framework, and the improved architectures are shown in Fig. 2.

LSTM, a specialized variant of the Recurrent Neural Network (RNN) [36], is known as one of the most widely used deep learning solutions for PPM. It is capable of solving the long-term dependency challenge inherent in RNN by remembering all trace prefix information and handling time series with extended intervals. As shown in Fig. 2(a), the input of the feature-selection-based prediction model based on LSTM (LSTM_FPM) is the trace prefix, and the hidden layer includes an LSTM layer and a linear layer.

Auto-encoded transition system (AETS) [35], as one of the latest application progresses of the Auto-encoder, is a natural language processing technique designed for process remaining time prediction. It enhances contextual understanding using the trace prefix vector encoding that obtained from the Auto-encoder as input for the multi-layer perception prediction model. The Auto-encoder is an unsupervised learning approach that includes encoder and decoder, which can be regarded as a dimensionality reduction technology. The input is taken as the learning objective to obtain a low-dimensional representation vector that retains the main information. The feature-selection-based prediction model of Auto-encoder (Auto-encoder_FPM) is shown in Fig. 2(b). It uses the trace prefix vector and other selected features of the last event as input, and its hidden layer is composed of linear layers.

Process Transformer (PT) [21] is a tailored process remaining time prediction model based on Transformer. Transformer [37] is a deep learning model based on the self-attention mechanism, including encoder and decoder. It effectively captures long-term dependencies and supports parallel computing, thereby enhancing training efficiency. Given the complexity of Transformer, its accuracy and performance theoretically outperform existing RNNs. To introduce this approach into the feature-selection-based prediction model, we re-implemented the PT architecture and adjusted the feature input position to accommodate the time series data of additional features. As shown in Fig. 2(c), the input of the modified prediction model (Transformer_FPM) is the trace prefix.

Note that the feature values in the trace prefix input to the prediction model in Fig. 2 are all encoded. Specifically, the CBoW encoding technique is applied to the active feature, the random vector encoding technique is used for other categorical features, and numerical features are standardized [24]. Taking Case 1 in Table 1 as an example, the e^4 trace prefix $tp^3(\sigma)$ is used as input to the prediction model, as illustrated in Fig. 3.

4.4 Prediction Model Updating Strategy

In real-life scenarios, business processes may evolve over time or change as enterprises scale up. As a result, the prediction models need to be continuously updated or reconstructed. To address this challenge, we propose three

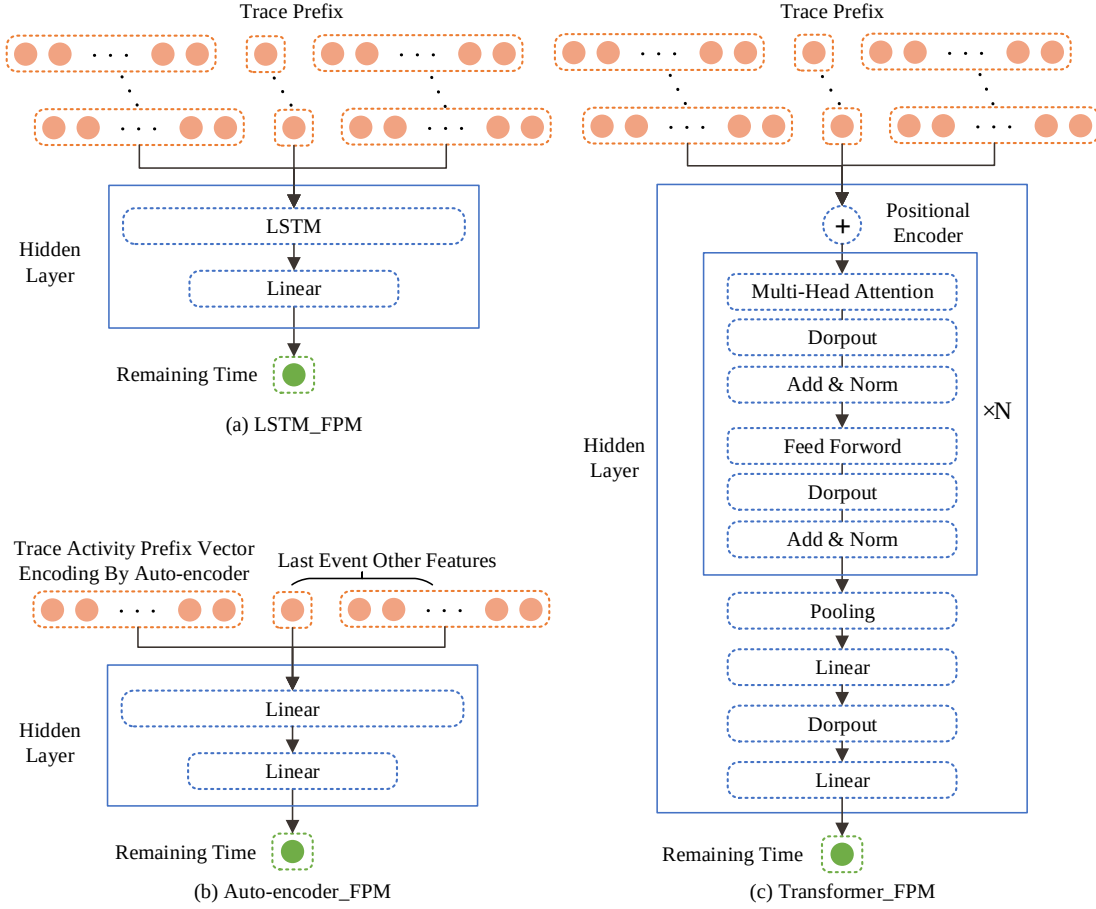


Fig. 2. Three Feature-selection-based Remaining Time Prediction Models.

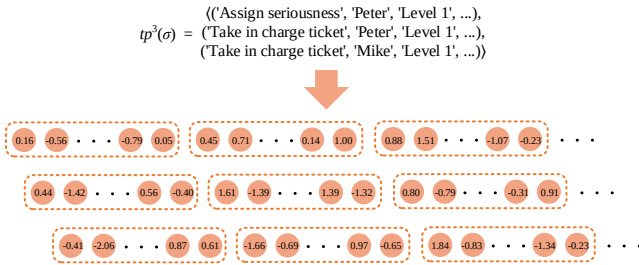


Fig. 3. An Example of Trace Prefix.

remaining time prediction model updating mechanisms, three training log reconstruction strategies for model updating, and a prediction model reconstruction strategy. The relationships among these three components are illustrated in Fig. 4. First, the trained feature-selection-based prediction model is used to predict the process instance remaining time online. As new process instances are continuously generated, update conditions are evaluated based on the selected update mechanism. When these conditions are met, the training log is reconstructed according to the chosen training log reconstruction strategy to either update or reconstruct the prediction model. Finally, the prediction model reconstruction strategy determines whether the feature combination selected from the training log matches the original feature combination. If the feature sets match, only

the prediction model is updated; otherwise, the prediction model undergoes reconstruction and retraining.

4.4.1 Prediction Model Updating Mechanism

In this paper, we propose three prediction model updating mechanisms as bellow.

- 1) **Period-based updating mechanism:** A fixed update period is defined based on domain knowledge, such as a month or a week, to update the prediction model accordingly. This approach enables the model to adapt to periodic or gradual changes driven by seasonality and enterprise development. For instance, the sales department can refine its marketing strategy for the upcoming month based on the latest market feedback.
- 2) **Quantity-based updating mechanism:** A case threshold is established based on the execution frequency of process instances. When the number of cases in the incremental event log reaches this threshold, the prediction model is updated. This approach ensures that sufficient new data is incorporated for each update, thereby avoiding frequent updates.
- 3) **Concept-drift-based updating mechanism:** Both period-based and quantity-based updating mechanisms rely on domain knowledge, making them sensitive to threshold settings, which may result in excessive or unnecessary updates to the prediction model. To solve this problem, a prediction model updating mechanism based on

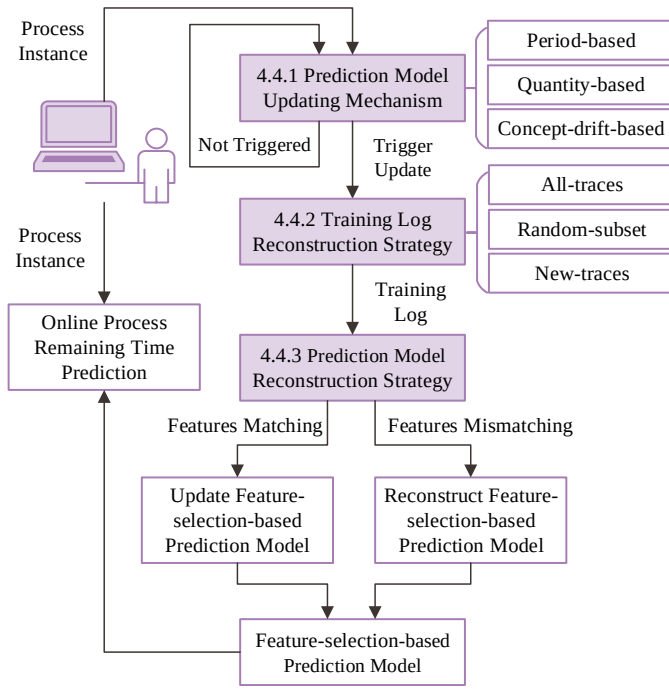


Fig. 4. Schematic Diagram of Prediction Model Updating and Reconstruction.

sliding windows is presented to update the prediction model when process changes or drifts are detected. It is argued that the detection of process concept drift is not limited to the control-flow perspective, but also involves other perspectives such as duration and resource. Therefore, we adopted the Kolmogorov-Smirnov Windowing (KSWIN) concept drift detection approach [38]. Specifically, KSWIN can monitor data distributions based on the KS statistical test by taking a fixed-length one-dimensional array as input. Considering the fact that the number of events in each trace is not always equal and the number of neural network units in the hidden layer of the prediction model is fixed, the complete trace is used as input for the prediction model construction, and the weight matrix in the hidden layer is used as the trace vector representation.

It is worth noting that besides the process remaining time prediction setting, the Period-based and Quantity-based updating mechanisms are more universally applied in various data incremental scenarios. In contrast, the Concept-drift-based updating mechanism is specifically designed for process remaining time prediction. It determines update opportunities by monitoring changes in process control-flow and data-flow, which significantly impacts the prediction accuracy.

4.4.2 Training Log Reconstruction Strategy

As for prediction model updating, three training log reconstruction strategies are given to adapt to event logs with diverse characteristics.

- **All-traces Strategy:** the entire historical event log and the incremental event log are used to update the prediction model to avoid historical information loss.

- **Random-subset Strategy:** the incremental event log and a randomly extracted subset of the historical event log are used to update the prediction model to improve the training efficiency.
- **New-traces Strategy:** only the incremental event log is used to update the prediction model to be focused on recent process instances.

After updating the prediction model, the incremental event log is added to the historical event log.

4.4.3 Prediction Model Reconstruction Strategy

Given the dynamic nature of business processes, the importance of each feature for the remaining time prediction may change. Therefore, it becomes essential to determine when to reconstruct the prediction model. In this paper, a prediction model reconstruction strategy based on feature combination matching is proposed. Before each updating, a new feature combination is re-selected using the feature selection strategy and matched with the original feature combination. If the top 50% most important features in the new feature combination are not included in the original feature combination, the prediction model is reconstructed rather than updated. Note that all historical event log are used as the training set for reconstructing the prediction model.

5 EXPERIMENTAL EVALUATION

This section evaluates the effectiveness of the incremental business process remaining time prediction framework. The event log and experimental settings are first introduced, and then the experimental results of the proposed three prediction models as well as the three prediction model updating mechanisms are qualitatively compared. Our approach is implemented in Python 3.7. The source code of our implementation and all experimental results are publicly available².

5.1 Data Set

In our experiments, nine real-life event logs from the 4TU Research Center³ are used as data sets.

- BPIC2015⁴ contains five event logs from five Dutch municipal governments.
- Sepsis⁵ is an event log from hospital sepsis cases.
- Helpdesk⁶ involves the ticket management process of the help desk in an Italian software company.
- Hospital⁷ contains events of the medical billing process in a dutch hospital.

2. <https://github.com/gn874682003/Incremental-Prediction-Framework>

3. <https://data.4tu.nl/>

4. <https://doi.org/10.4121/uuid:31a308ef-c844-48da-948c-305d167a0ec1>

5. <https://doi.org/10.4121/uuid:0c60edf1-6f83-4e75-9367-4c63b3e9d5bb>

6. <https://doi.org/10.4121/uuid:0c60edf1-6f83-4e75-9367-4c63b3e9d5bb>

7. <https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460>

- Road_Traffic⁸ involves the road traffic fine management process in an information system.

Table 2 shows the basic statistical overview of these event logs, where the MCL represents the mean length of all cases in each experimental log.

TABLE 2
Statistics of event logs

Data Set	Cases	Events	Activities	Variants	MCL
BPIC2015_1	1199	52217	398	1170	44
BPIC2015_2	832	44354	410	828	53
BPIC2015_3	1409	59681	383	1349	42
BPIC2015_4	1053	47293	356	1049	45
BPIC2015_5	1156	59083	389	1153	51
Sepsis	1050	15214	16	846	14
Helpdesk	4580	21348	14	226	5
Hospital	100000	451359	18	1020	5
Road_Traffic	150370	561470	14	231	4

5.2 Experimental Setting

Hyper-parameters of the remaining time prediction model are tuned using the following ranges in [24]: (1) learning rate: $\{10^{-2}, 10^{-3}, 10^{-4}\}$; (2) batch size: $\{20, 50, 100, 150\}$; (3) the number of training epoch: $\{100, 200, 300, 400, 500\}$; and (4) optimization algorithm: Adam. The number of hidden layer nodes is fine-tuned for each event log and prediction model to optimize performance.

In the experimental evaluation of the feature-selection-based prediction model in Section 5.3, the training set and test set are randomly divided by 8:2 ratio. As for the prediction model updating mechanism evaluation in Section 5.4, the event log is divided into the training set and the test set in a ratio of 1:1 by year according to the completion time of traces. The training set is used as the historical event log for training the prediction model. The test set is used to simulate the incremental process to generate the incremental event log, which is used to test the updating mechanism and update the prediction model. During the prediction model updating process, the epoch value is set to 20, and the random extraction ratio of the training set is 20%. Without loss of generality, period-based updating mechanism is set to update the prediction model on a monthly basis (yearly or weekly settings are also feasible). Quantity-based updating mechanism is set to update the prediction model per 100 traces, based on the batch size of the prediction model. However, for the two large-scale event logs, i.e., Hospital and Road_Traffic, the update threshold is adjusted to 1000 traces accordingly.

To evaluate the accuracy of feature selection strategy, the Mean Absolute Error (MAE) is applied and computed as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y - \hat{y}| \quad (1)$$

where, n represents the number of samples, y represents the true value and $\hat{y}$ is the prediction value. A smaller MAE value indicates a higher prediction accuracy.

8. <https://doi.org/10.4121/uuid:76c46b83-c930-4798-a1c9-4be94dfeb741>

5.3 Results of Feature-selection-based Prediction Models

Comparative experiments are conducted using three input feature sets: *Activity*, all attributes, and the effective feature combination (EFC) selected by the feature selection strategy, denoted as #Activity, #All, and EFC, respectively. The prediction model is constructed using the LightGBM algorithm [39] applied in the feature selection strategy. The experimental results are shown in Table 3, where FeaNum means the number of features.

TABLE 3
Results of Different Feature Combinations (FeaNum means the number of selected features.)

Data sets	Remaining time (MAE: Day)			
	#Activity	#All (FeaNum)	EFC (FeaNum)	Prefix5
BPIC2015_1	39.96	28.10(22)	32.47(5)	27.47
BPIC2015_2	78.94	73.76(22)	68.73(9)	67.79
BPIC2015_3	22.93	20.83(22)	18.48(7)	16.73
BPIC2015_4	63.65	52.07(22)	49.98(8)	52.30
BPIC2015_5	49.98	43.14(22)	38.94(14)	38.90
Sepsis	39.78	50.09(35)	42.45(11)	45.73
Helpdesk	6.57	5.01(18)	4.78(3)	3.12
Hospital	57.16	35.79(24)	35.80(10)	29.33
Road_Traffic	217.69	197.13(17)	194.98(10)	41.15

As can be seen from Table 3, #All reduces the prediction error compared with #Activity, indicating that richer input information helps capture differences in business process execution details. However, there is an exception, i.e., the Sepsis Log. One possible explanation is that it has the negative features with greater impact. In addition, both the MAE values and the number of features in EFC are lower than those in #Activity and #All, which shows that feature selection strategy effectively identifies an optimal feature combination for model training.

In the feature selection strategy, the input consists of feature values from the last event. However, the trace prefix also plays an important role in remaining time prediction. To illustrate the positive impact of prefix information in the selected feature combination EFC, a comparative experiment Prefix5 is conducted, where the prefix length of the prediction model input is set to 5, as indicated in the last column of Table 3. The results show a significant improvement in prediction accuracy after adding the trace prefix. However, there are two exceptions, i.e., BPIC2015_4 and Sepsis logs. For BPIC2015_4, the influence of the trace prefix appears to be comparatively minor. For Sepsis, MAE values of Prefix5 and EFC are both larger than #Activity. One possible explanation is that the data distribution deviation leads to model over-fitting. Based on the experimental results, it is concluded that the proposed strategy is able to obtain effective features to improve the prediction accuracy in most cases. Additionally, the prefix information of selected features positively impacts remaining time prediction, and the feature combination can be extended to other learning models with the ability to process time series data.

To demonstrate the effectiveness of the feature-selection-based remaining time prediction model, the LSTM (with activity feature as input), PT [21], and AETS [35] are used as

TABLE 4
Results of Feature-selection-based Prediction Model

Data sets	Remaining time (MAE: Day)					
	LSTM	LSTM_FPM	PT [21]	Transformer_FPM	AETS [35]	Auto-encoder_FPM
BPIC2015_1	29.26	26.44	24.71	26.52	37.88	36.33
BPIC2015_2	71.94	66.69	68.89	68.21	57.89	56.85
BPIC2015_3	19.19	18.40	17.71	16.80	19.06	17.70
BPIC2015_4	52.63	49.02	49.97	54.58	38.31	38.14
BPIC2015_5	36.86	35.85	36.04	36.45	45.14	44.01
Sepsis	31.78	30.64	26.07	27.46	35.39	35.07
Helpdesk	6.29	4.59	5.50	1.18	8.63	6.37
Hospital	63.30	43.31	42.79	2.98	49.51	50.40
Road_Traffic	264.12	276.12	112.21	10.91	304.46	310.57

TABLE 5
Statistics of Feature-selection-based Prediction Model (Params refers to the number of parameters in the prediction model, and Time denotes the training time of the prediction model, measured in seconds.)

Data sets	LSTM		LSTM_FPM		PT [21]		Transformer_FPM		AETS [35]		Auto-encoder_FPM	
	Params	Time(s)	Params	Time(s)	Params	Time(s)	Params	Time(s)	Params	Time(s)	Params	Time(s)
BPIC2015_1	32061	118s	38373	132s	37309	215s	103566	848s	1340241	1104s	1747641	1106s
BPIC2015_2	32397	113s	45853	206s	38857	190s	127541	840s	1199675	803s	1884323	897s
BPIC2015_3	31641	95s	41421	167s	37597	204s	159101	1008s	2033511	1032s	2952807	1341s
BPIC2015_4	30885	125s	42477	137s	36337	169s	102520	754s	1760397	744s	2678445	954s
BPIC2015_5	31809	160s	55365	180s	38893	235s	178832	1191s	2081528	1001s	4147536	1300s
Sepsis	11101	40s	22729	82s	26581	61s	65505	240s	392842	163s	662842	168s
Helpdesk	7185	72s	8989	171s	20389	109s	20945	625s	185303	117s	218759	119s
Hospital	11141	1972s	24321	2351s	27805	2151s	231789	17963s	925706	3496s	1648010	3691s
Road_Traffic	7137	1661s	16893	2296s	20461	2694s	125136	22527s	142282	2609s	235282	2683s

baseline models, and three corresponding feature-selection-based prediction models (FPM), i.e., LSTM_FPM, Transformer_FPM and Auto-encoder_FPM, are compared. The experimental results are shown in Table 4, and Table 5 displays the training time for different models and their total number of parameters.

According to Table 4, we have the following observations. Firstly, compared to the LSTM prediction model, the LSTM_FPM generally shows superior performance except for the Road_Traffic log. For the Road_Traffic log, because of its large scale, the LSTM neural network is limited in fitting the multi-feature input. Similarly, the comparison between AETS and Auto-encoder_FPM shows that the prediction accuracy decreases in large-scale logs, e.g., Hospital and Road_Traffic.

Secondly, for Helpdesk, Hospital, and Road_Traffic logs, MAE values of the Transformer_FPM are much lower than those of the PT, indicating that key features provide more effective information for remaining time prediction, and the Transformer demonstrates strong understanding and representation ability to capture this information. However, for BPIC2015 and Sepsis logs with an excessive number of variants, these two models demonstrate similar performance. One possible explanation is that the input of the PT also contains rich time information.

As shown in Table 5, the number of parameters (denoted as *Params*) and the training time of the three feature-selection-based models exceed those of their corresponding baseline models. This is because processing additional effective features requires more neural units, which increases both the model parameters and the training time needed for proper data fitting. Furthermore, across all event logs,

LSTM has fewer parameters than that of PT due to the Transformer's structure is more complex. However, despite AETS using only fully connected layers, it has the highest parameter count. This is because it constructs separate prediction models for each trace prefix length. Nevertheless, since each individual prediction model has a relatively simple structure, the overall training time remains manageable. A similar trend is observed across all three feature-selection-based models.

In summary, the experimental results indicate that feature-selection-based models, enriched with effective information, outperform state-of-the-art prediction models. Among the six prediction models, the Transformer-based prediction model demonstrates the best fitting and generalization capabilities. In addition, the increase in parameter count and training time remains within an acceptable range.

5.4 Results of Prediction Model Updating Mechanism

By taking the period-based updating mechanism as an example, we first introduce how to find the most suitable training log reconstruction strategy for each experimental log. To be specific, three feature-selection-based prediction models are used as prediction models, and the incremental results of three training log reconstruction strategies are tested and the results are given in Table 6.

Based on the experimental testing, for Sepsis and BPIC2015 series logs, the All-traces strategy is suitable for logs with a great number of variants to ensure the diversity of samples. The Random-subset strategy performs best for the Helpdesk log. Although it cannot guarantee the optimal result for the Auto-encoder_FPM, it is very close to the best result. For Hospital and Road_Traffic logs, the New-

TABLE 6
Results of Period-based Updating Mechanism Using Different Training Log reconstruction Strategies

Data sets	Remaining time (MAE: Day)								
	LSTM_FPM			Transformer_FPM			Auto-encoder_FPM		
	All-traces	Random-subset	New-traces	All-traces	Random-subset	New-traces	All-traces	Random-subset	New-traces
BPIC2015_1	45.72	48.75	53.47	41.81	47.95	49.92	63.93	82.84	91.48
BPIC2015_2	79.28	82.73	88.30	71.94	80.21	89.71	111.99	125.80	131.55
BPIC2015_3	23.43	24.85	26.95	6.81	7.25	12.17	25.66	29.63	32.50
BPIC2015_4	51.24	50.17	50.08	14.12	17.64	23.65	50.29	55.81	54.93
BPIC2015_5	45.79	49.17	56.66	43.90	48.63	55.47	72.09	82.08	82.61
Sepsis	30.26	30.88	33.06	11.82	23.75	33.00	33.61	33.32	32.40
Helpdesk	4.40	4.34	4.80	1.67	1.16	1.26	5.56	5.74	5.94
Hospital	38.89	36.74	33.76	4.92	5.74	4.18	60.34	41.44	38.82
Road_Traffic	267.17	278.88	80.96	25.76	17.88	19.39	483.62	340.41	148.95

traces strategy can effectively improve efficiency and ensure prediction accuracy for these large-scale logs. Based on the above observations, we adopt the best training log reconstruction strategy for each experimental log in subsequent experiments to update the model.

To illustrate the effectiveness of prediction model updating mechanisms based on the incremental event log, the original training approach, i.e., the non-increment, is employed as a baseline, and the state-of-the-art technique and three prediction model updating mechanisms are compared. To the best of our knowledge, no incremental prediction approach has been specifically designed for process remaining time prediction. Therefore, we integrate the concept drift detection technique based on control flow (denoted as CF-drift), originally used by Pauwels *et al.* [29] for next event incremental prediction, into our proposed framework. This allows us to evaluate the applicability of different updating mechanisms. Since Pauwels *et al.* [29] do not provide detailed configuration information for their concept drift detection technique, we use the drift points identified in six event logs, which correspond to our datasets, as update triggers for experimental testing. The experimental results are shown in Table 7, where Average Runtime refers to the average execution time required to update the prediction model. Taking the period-based updating mechanism as an example, Fig. 5 visualizes MAE values of the three feature-selection-based models and the non-increment model for each period, where the solid line represents prediction results of the non-increment model, and the dotted line represents prediction results of the model using period-based updating mechanism. For page limits, experimental results of the quantity-based updating mechanism and the concept-drift-based updating mechanism are not given in the paper. Please refer to the whole experimental results online⁹.

According to Fig. 5, the prediction accuracy of the non-increment model fluctuates obviously with time. However, three prediction models using the period-based updating mechanism can capture the changes of new instances and lead to lower MAE values for almost every period. As can be seen from Table 7, the prediction model achieves better prediction accuracy by applying the three proposed prediction

model updating mechanisms than the non-incremental one for all event logs, which indicates that the continuous updating of the prediction model effectively handles dynamic process changes and ensures high prediction accuracy. However, there are two exceptions, i.e., the LSTM_FPM of the BPIC2015_4 log and the Auto-encoder_FPM of the Sepsis log. One possible explanation is that these two models are limited by their network structure, which may lead to instability of the prediction model when logs contain a large number of variants. In addition, the experimental results of CF-drift across the six event logs are inferior to those of the three proposed update mechanisms and do not consistently outperform the non-increment approach. This indicates that the concept drift detection technique based on control flow is not directly applicable to incremental remaining time prediction.

Among the three prediction models, the Auto-encoder_FPM has the largest prediction error for all event logs. Although it has certain advantages in extracting and identifying activity prefix information, the learning and prediction ability of the model is quite limited because of the simple network structure. In addition, its average runtime is the longest, because each update needs to learn the prefix representation again. After applying the prediction model updating mechanism, the improvement of prediction accuracy of the LSTM_FPM is not as obvious as those of the Auto-encoder_FPM and the Transformer_FPM for logs with relatively long traces, such as BPIC2015 series logs. The underlying reason is that as the input sequence length increases, the training difficulty of the LSTM network also increases, making it difficult to capture global dependencies. Hopefully, its updating efficiency is the highest compared with the other two models. In contrast, the Transformer_FPM using any of the three prediction model updating mechanisms can achieve the lowest MAE value for all experimental logs, indicating its strong sequence modeling and information perception ability. However, there is an exception, i.e., the LSTM_FPM with the quantity-based updating mechanism in BPIC2015_5 log. One possible explanation is that the quantity-based updating mechanism is not suitable for this event log, demonstrating that an inappropriate updating mechanism cannot improve the prediction accuracy.

In general, the quantity-based updating mechanism

9. <https://github.com/gn874682003/Incremental-Prediction-Framework/tree/main/Result>

TABLE 7

Results of Prediction Model Updating Mechanisms (The All-traces strategy is used in Sepsis and BPIC2015 series logs, the Random-subset strategy is used in the Helpdesk log, and the New-traces strategy is used in Hospital and Road_Traffic logs.)

Data sets	Models	Remaining time (MAE: Day)					Average Runtime(s)
		Non-increment	Period	Quantity	Concept-drift	CF-drift [29]	
BPIC2015_1	LSTM_FPM	52.55	46.01	50.08	50.26	51.48	30
	Transformer_FPM	49.30	40.37	46.89	48.38	51.20	103
	Auto-encoder_FPM	159.27	64.81	89.26	98.75	97.48	998
BPIC2015_2	LSTM_FPM	86.23	79.92	82.11	83.74	88.03	20
	Transformer_FPM	85.20	67.56	80.48	72.90	87.77	79
	Auto-encoder_FPM	186.20	111.91	147.19	151.93	191.50	657
BPIC2015_3	LSTM_FPM	26.84	24.10	25.46	24.85	25.38	32
	Transformer_FPM	12.06	6.81	9.71	6.58	12.06	109
	Auto-encoder_FPM	37.25	25.66	32.38	23.85	33.55	969
BPIC2015_4	LSTM_FPM	51.84	51.81	53.30	54.94	50.79	29
	Transformer_FPM	23.46	14.82	19.56	18.03	22.50	103
	Auto-encoder_FPM	114.44	50.29	56.44	53.68	85.13	874
BPIC2015_5	LSTM_FPM	58.81	46.53	52.22	57.63	54.26	71
	Transformer_FPM	65.82	44.77	54.33	55.37	57.52	151
	Auto-encoder_FPM	104.49	72.09	92.84	99.42	116.53	1037
Sepsis	LSTM_FPM	33.52	30.21	29.88	30.29	-	7
	Transformer_FPM	33.12	11.68	13.55	18.84	-	21
	Auto-encoder_FPM	31.63	33.61	34.00	30.62	-	241
Helpdesk	LSTM_FPM	4.42	4.34	4.28	4.36	4.54	2
	Transformer_FPM	1.56	1.32	1.35	1.20	1.44	5
	Auto-encoder_FPM	6.44	5.91	6.02	6.13	6.24	28
Hospital	LSTM_FPM	56.01	38.08	27.44	26.35	-	8
	Transformer_FPM	12.83	4.51	4.41	4.00	-	35
	Auto-encoder_FPM	69.69	39.19	30.81	29.01	-	385
Road_Traffic	LSTM_FPM	326.03	93.96	53.60	51.00	-	2
	Transformer_FPM	30.92	19.39	20.55	15.58	-	87
	Auto-encoder_FPM	617.06	148.95	83.95	69.82	-	32

leads to the largest prediction error for all experimental logs among the three prediction model updating mechanisms, which is caused by the lack of domain knowledge to set the appropriate threshold. The period-based updating mechanism achieves the minimum MAE value for the Sepsis and BPIC2015 series logs, which indicates that the process changes have time periodicity for these complex or flexible business processes. Therefore, a reasonable period according to domain knowledge can effectively improve the accuracy and stability of the prediction model. However, the concept-drift-based updating mechanism obtains slightly better prediction results for the BPIC2015_3 log, which shows that the concept drift detection technique can precisely capture process changes. In addition, the concept-drift-based updating mechanism also achieves the best prediction results for the Helpdesk, Hospital, and Road_Traffic logs. This indicates that for process logs with large throughput and a relatively standard distribution, the concept-drift-based updating mechanism updates the prediction model after process information changes significantly to ensure in-time updates.

Based on the above analysis, it is recommended to use Transformer_FPM, with its superior data processing capabilities, to predict process remaining time within the proposed framework. For business processes with flexible variants and small-scale data, the period-based updating mechanism combined with the All-traces training log construction strategy is suitable to ensure timely and effective updates. Conversely, for businesses with large-scale data and relatively stable process modes, the concept-drift-based updating mechanism, along with the New-traces training

log construction strategy, is more appropriate to maintain update efficiency. In addition, users with professional knowledge can choose the appropriate update mechanism and set a reasonable threshold based on their experience, while ordinary users are recommended to use the concept-drift-based update mechanism because it suits a more general and extensive application setting.

5.5 Results of Prediction Model Reconstruction Strategy

Based on the Section 5.4 experimental analysis, we selected the Transformer_FPM model in combination with the period-based updating mechanism to evaluate the prediction model reconstruction strategy, and the results are shown in Table 8. It can be seen that prediction accuracy is significantly improved using the prediction model reconstruction strategy, and the number of reconstructions is relatively reasonable. However, Helpdesk and Road_Traffic logs are exceptions. For Helpdesk log, the MAE value of reconstruction slightly surpassed the original result. Since this process is relatively standard, the reconstruction has little impact. Moreover, minor variations in neural network training results are expected. For Road_Traffic log, no reconstruction is triggered. This observation suggests that process changes not affect the ranking of the importance features. Furthermore, it validates the effectiveness and applicability of the prediction model reconstruction strategy.

6 THREADS TO VALIDITY

In the following, we discuss the main threats that may affect the validity of our approach.

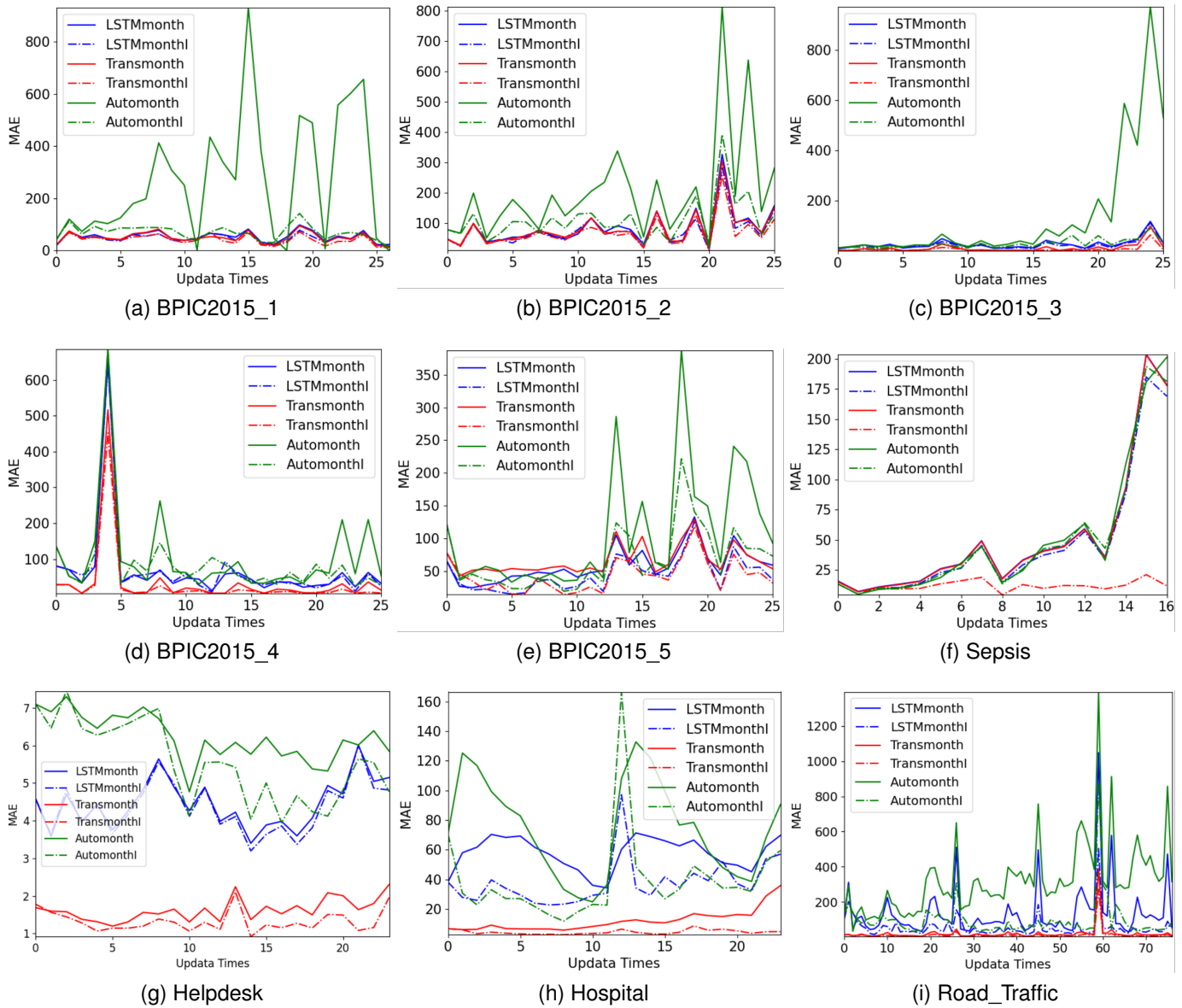


Fig. 5. Results of Period-based Updating Mechanism.

TABLE 8
Results of Prediction Model Reconstruction Strategy (Number means how many times the prediction model has been reconstructed.)

Data sets	Remaining time (MAE: Day)	
	Results in Table 7	Reconstruct (Number)
BPIC2015_1	40.37	18.03 (3)
BPIC2015_2	67.56	25.49 (2)
BPIC2015_3	6.81	6.69 (1)
BPIC2015_4	14.82	13.57 (1)
BPIC2015_5	44.77	13.05 (1)
Sepsis	11.68	7.19 (9)
Helpdesk	1.20	1.21 (1)
Hospital	4.51	4.01 (1)
Road_Traffic	19.39	19.39 (0)

- The period-based and quantity-based updating mechanisms rely on domain knowledge to set reasonable thresholds, and therefore, their effectiveness

is limited by new unknown business scenarios.

- The concept-drift-based updating mechanism depends on the training effectiveness and trace representation ability of the remaining time prediction model. Therefore, the data extraction and representation abilities of the selected prediction model, along with its suitability for event logs, directly impact the effectiveness of the updating mechanism in monitoring business dynamic changes.
- The effectiveness of the feature-selection-based prediction model depends on the presence of other effective features in the event log, and therefore, the approach may perform worse if the event log does not contain enough effective features.
- The effectiveness of the incremental framework in addressing dynamic changes in process instances depends on the regularity of data distribution shifts in new instances, which inherently introduces a lag.

As a result, this approach has limitations in handling sudden changes and unforeseen anomalies in business processes.

7 CONCLUSION

Existing process prediction models lack an effective updating mechanism to support incremental event logs especially for dynamically changing processes. In addition, continuous prediction model updating requires the prediction model to effectively capture these changes. To address the above challenges, we propose a framework for incremental business process remaining time prediction. Three models are implemented into the feature-selection-based prediction framework, and nine real-life event logs are used to simulate the prediction model updating process. Experimental results demonstrate the usefulness of the proposed approach and show its improved prediction accuracy.

With the dynamic change in business processes, the historical event log may become uninformative or even counterproductive to the current model training. Therefore, our future work may focus on the application of incremental learning algorithms to balance learning plasticity and memory stability. In addition, the proposed concept-drift-based updating mechanism depends on the reliability of the prediction model, and it is difficult to decide the appropriate updating time because of insufficient model training in small-scale event logs. Therefore, other concept drift techniques can be investigated in our future work.

REFERENCES

- [1] W. Aalst, van der, *Process mining : discovery, conformance and enhancement of business processes*. Germany: Springer, 2011.
- [2] C. Liu, H. Duan, Q. Zeng, M. Zhou, F. Lu, and J. Cheng, "Towards comprehensive support for privacy preservation cross-organization business process mining," *IEEE Transactions on Services Computing*, vol. 12, no. 4, pp. 639–653, 2016.
- [3] N. Harane and S. Rath, "Comprehensive survey on deep learning approaches in predictive business process monitoring," *Modern Approaches in Machine Learning and Cognitive Science: A Walk-through*, pp. 115–128, 2020.
- [4] X. Sun, W. Hou, Y. Ying, and D. Yu, "Remaining time prediction of business processes based on multilayer machine learning," in *2020 IEEE International Conference on Web Services (ICWS)*, 2020, pp. 554–558.
- [5] M. Harl, S. Weinzierl, M. Stierle, and M. Matzner, "Explainable predictive business process monitoring using gated graph neural networks," *Journal of Decision Systems*, vol. 29, no. sup1, pp. 312–327, 2020.
- [6] C. Hsieh, C. Moreira, and C. Ouyang, "Interpreting process predictions using a milestone-aware counterfactual approach," *arXiv e-prints*, pp. arXiv:2107, 2021.
- [7] J. Evermann, J.-R. Rehse, and P. Fettke, "Predicting process behaviour using deep learning," *Decision Support Systems*, vol. 100, pp. 129–140, 2017, smart Business Process Management. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167923617300635>
- [8] A. Bolt and M. Sepúlveda, "Process remaining time prediction using query catalogs," in *International Conference on Business Process Management*. Springer, 2013, pp. 54–65.
- [9] J. Roider, A. Nguyen, D. Zanca, and B. M. Eskofier, "Assessing the performance of remaining time prediction methods for business processes," *IEEE Access*, 2024.
- [10] A. E. Márquez-Chamorro, M. Resinas, and A. Ruiz-Cortés, "Predictive monitoring of business processes: a survey," *IEEE Transactions on Services Computing*, vol. 11, no. 6, pp. 962–977, 2017.
- [11] I. Teinemaa, M. Dumas, M. L. Rosa, and F. M. Maggi, "Outcome-oriented predictive process monitoring: review and benchmark," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 13, no. 2, pp. 1–57, 2019.
- [12] D. A. Neu, J. Lahann, and P. Fettke, "A systematic literature review on state-of-the-art deep learning methods for process prediction," *Artificial Intelligence Review*, pp. 1–27, 2021.
- [13] I. Verenich, M. Dumas, M. L. Rosa, F. M. Maggi, and I. Teinemaa, "Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 4, pp. 1–34, 2019.
- [14] A. Pika, W. M. Van Der Aalst, C. J. Fidge, A. H. Ter Hofstede, and M. T. Wynn, "Profiling event logs to configure risk indicators for process delays," in *International Conference on Advanced Information Systems Engineering*. Springer, 2013, pp. 465–481.
- [15] W. M. Van der Aalst, M. H. Schonenberg, and M. Song, "Time prediction based on process mining," *Information systems*, vol. 36, no. 2, pp. 450–475, 2011.
- [16] A. Rogge-Solti and M. Weske, "Prediction of remaining service execution time using stochastic petri nets with arbitrary firing delays," in *International conference on service-oriented computing*. Springer, 2013, pp. 389–403.
- [17] N. Navarin, B. Vincenzi, M. Polato, and A. Sperduti, "Lstm networks for data-aware remaining time prediction of business process instances," in *2017 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2017, pp. 1–7.
- [18] S. Weinzierl, S. Zilker, S. Dunzer, and M. Matzner, "Machine learning in business process management: A systematic literature review," *Expert Systems with Applications*, p. 124181, 2024.
- [19] N. Tax, I. Verenich, M. L. Rosa, and M. Dumas, "Predictive business process monitoring with lstm neural networks," in *International Conference on Advanced Information Systems Engineering*. Springer, 2017, pp. 477–492.
- [20] M. Camargo, M. Dumas, and O. González-Rojas, "Learning accurate lstm models of business processes," in *International Conference on Business Process Management*. Springer, 2019, pp. 286–302.
- [21] Z. A. Bukhsh, A. Saeed, and R. M. Dijkman, "Processtransformer: Predictive business process monitoring with transformer network," *arXiv preprint arXiv:2104.00721*, 2021.
- [22] L. T. Duong, L. Travé-Massuyès, A. Subias, and C. Merle, "Remaining cycle time prediction with graph neural networks for predictive process monitoring," in *Proceedings of the 2023 8th International Conference on Machine Learning Technologies*, 2023, pp. 95–101.
- [23] K. Amiri Elyasi, H. van der Aa, and H. Stuckenschmidt, "Pgtnet: A process graph transformer network for remaining time prediction of business process instances," in *International Conference on Advanced Information Systems Engineering*. Springer, 2024, pp. 124–140.
- [24] N. Guo, C. Liu, C. Li, Q. Zeng, C. Ouyang, Q. Liu, and X. Lu, "Explainable and effective process remaining time prediction using feature-informed cascade prediction model," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 949–962, 2024.
- [25] J. Cao, N. Wang, S. Qian, and W. Guan, "When to invoke a prediction service for business process monitoring?" *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 3061–3074, 2023.
- [26] M. Maisenbacher and M. Weidlich, "Handling concept drift in predictive process monitoring," in *2017 IEEE International Conference on Services Computing (SCC)*. IEEE, 2017, pp. 1–8.
- [27] L. Baier, J. Reimold, and N. Kühl, "Handling concept drift for predictions in business process mining," in *2020 IEEE 22nd Conference on Business Informatics (CBI)*, vol. 1. IEEE, 2020, pp. 76–83.
- [28] W. Rizzi, C. Di Francescomarino, C. Ghidini, and F. M. Maggi, "How do i update my model? on the resilience of predictive process monitoring models to change," *Knowledge and Information Systems*, vol. 64, no. 5, pp. 1385–1416, 2022.
- [29] S. Pauwels and T. Calders, "Incremental predictive process monitoring: The next activity case," in *Business Process Management*, A. Polyvyanyy, M. T. Wynn, A. Van Looy, and M. Reichert, Eds. Cham: Springer International Publishing, 2021, pp. 123–140.
- [30] V. Pasquadibisceglie, A. Appice, G. Castellano, and D. Malerba, "Darwin: An online deep learning approach to handle concept drifts in predictive process monitoring," *Engineering Applications of Artificial Intelligence*, vol. 123, p. 106461, 2023.
- [31] A. S. Mangat and S. Rinderle-Ma, "Next-activity prediction for non-stationary processes with unseen data variability," in *Interna-*

tional Conference on Enterprise Design, Operations, and Computing. Springer, 2022, pp. 145–161.

- [32] Y. Goldberg and O. Levy, “word2vec explained: deriving mikolov et al.’s negative-sampling word-embedding method,” *arXiv preprint arXiv:1402.3722*, 2014.
- [33] H. Y. Kim, N. Balasubramanian, and B. Kang, “On initializing transformers with pre-trained embeddings,” *arXiv preprint arXiv:2407.12514*, 2024.
- [34] A. Graves, “Long short-term memory,” *Supervised sequence labelling with recurrent neural networks*, pp. 37–45, 2012.
- [35] W. Ni, M. Yan, T. Liu, and Q. Zeng, “Predicting remaining execution time of business process instances via auto-encoded transition system,” *Intelligent Data Analysis*, vol. 26, no. 2, pp. 543–562, 2022.
- [36] A. Sherstinsky, “Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network,” *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020.
- [37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [38] C. Raab, M. Heusinger, and F.-M. Schleif, “Reactive soft prototype computing for concept drift streams,” *Neurocomputing*, vol. 416, pp. 340–351, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231220305063>
- [39] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” *Advances in neural information processing systems*, vol. 30, 2017.



Na Guo received the MS degree in computer science and technology from Shandong University of Technology, Zibo, China, in 2021. She is current working towards the PhD degree in Shandong University of Technology. Her research interests are in the areas of process mining and process predictive monitoring.



Cong Liu received the PhD degree in the Department of Mathematics and Computer Science, Eindhoven University of Technology, 2019. He is an invited full professor in the NOVA Information Management School, Nova University of Lisbon. His research interests are in the areas of process mining, business process management, and artificial intelligence.



Qi Mo received the Ph.D. degree in system analysis and integration from Yunnan University, Kunming, China, in 2015. He is currently a Lecturer with the School of Software, Yunnan University. His research interests are in formal methods and business process management.



papers in prestigious journals and conferences. Dr. Cao has won 10 ministerial or provincial level scientific and technological achievements rewards. Currently, he is the distinguished member of CCF and the senior member of IEEE.

Jian Cao is currently a tenured professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. He is also the director of research institute of network computing and service computing. Dr. Cao's research interests include intelligent data analytics, service computing, collaborative computing and software engineering. Besides national and provincial government research grants, his research is also supported by many industry partners. He has published more than 300 research



Ouyang Chun received the PhD degree in Computer Systems Engineering from University of South Australia. She is currently a associate professor with Queensland University of Technology, Brisbane, Australia. Her research interests are in the areas of data-driven process analytics, process automation, and recently explainable predictive analytics.



Xixi Lu received the MS and PhD degree in the Department of Mathematics and Computer Science, Eindhoven University of Technology. She is a Assistant Professor in Utrecht University, in 2019. Her research interests are in the areas of process mining and data science.



Qingtian Zeng received the BS degree and the MS degree in computer science from Shandong University of Science and Technology, Taian, China, in 1998 and 2001 respectively, and the PhD degree in computer software and theory from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2005. He is currently a Professor with Shandong University of Science and Technology, Qingdao, China. His research interests are Petri nets, process mining, and knowledge management.